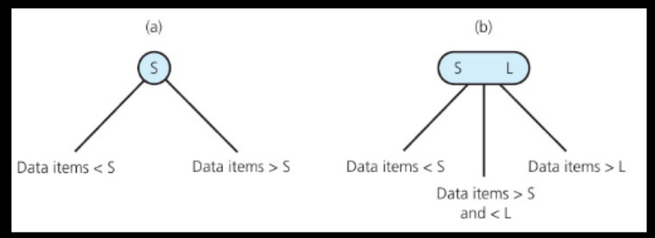
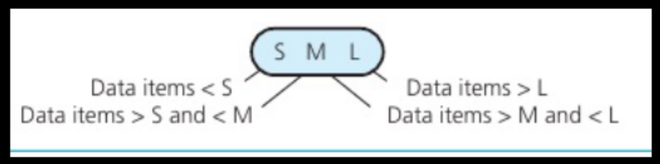
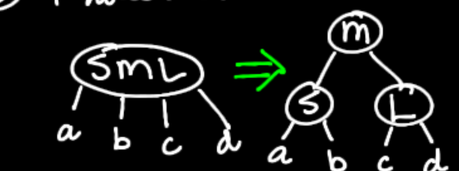
① 2-3-4 Tree

1 item – 2 children – 2-node
2 items – 3 children – 3-node
3 items – 4 children – 4-node
preemptive – split any 4-node on the insertion path
non preemptive – do not split on the insertion path

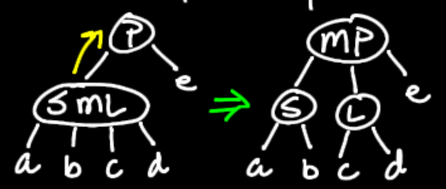


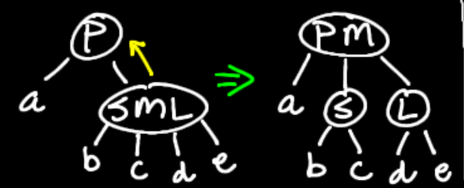


② 4-node root

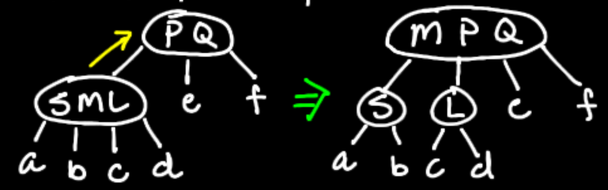


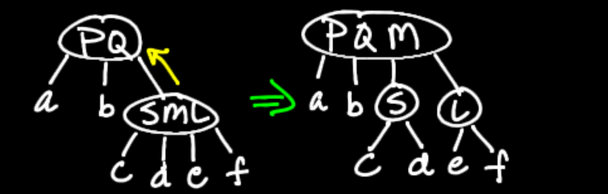
4-node w/ 2 node parent

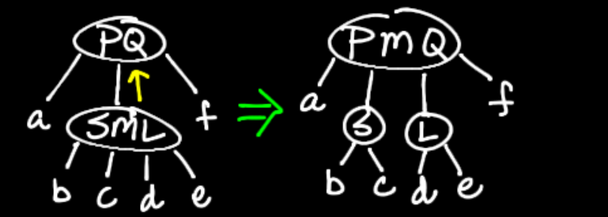




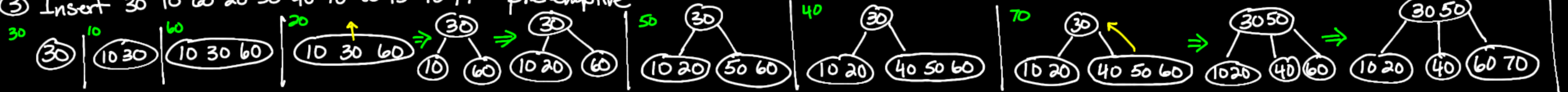
4-node w/ 3 node parent

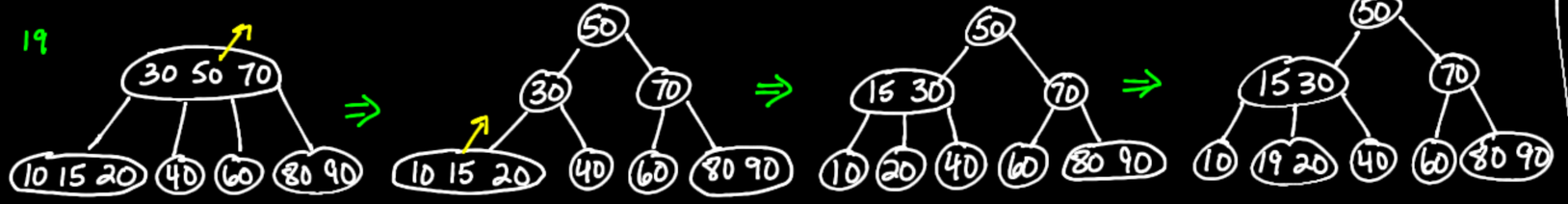




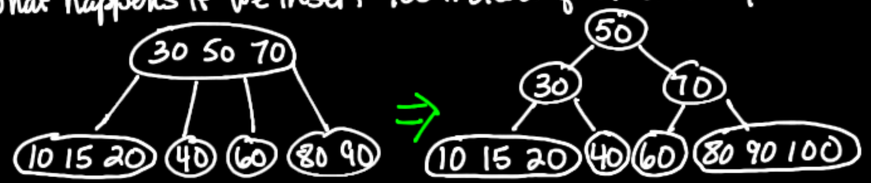


③ Insert 30 10 60 20 50 40 70 80 15 90 19 Pre-emptive

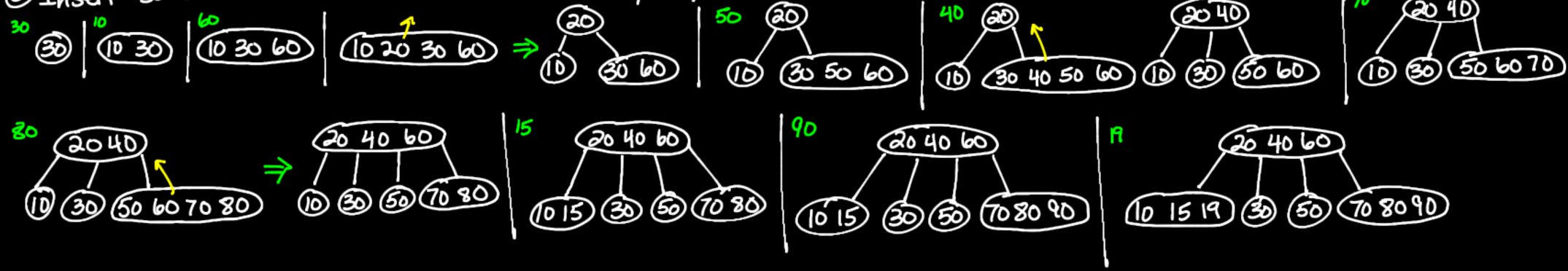




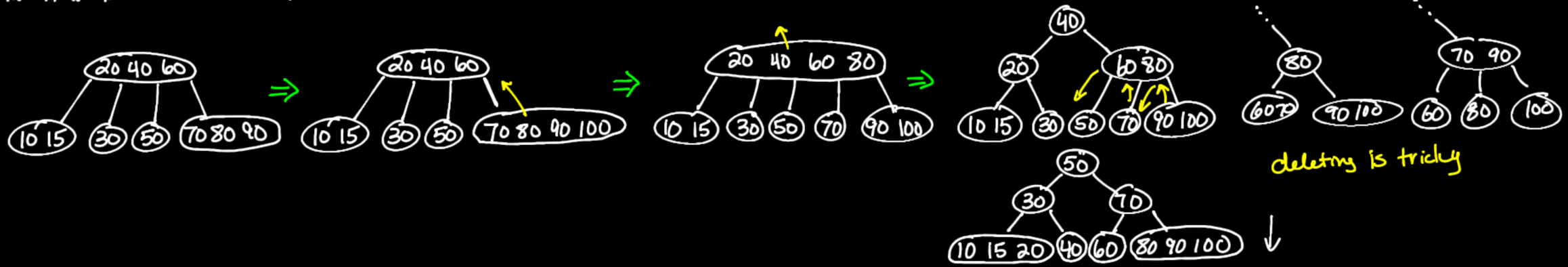
Let steps back for a moment
what happens if we insert 100 instead of 19 at this point?



⑤ Insert 30 10 60 20 50 40 70 80 15 90 19 non-preemptive



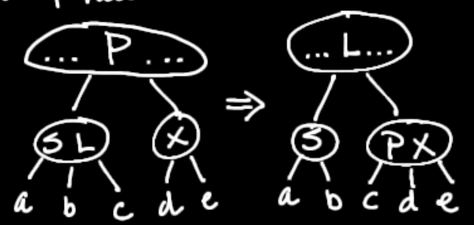
What if we insert 100 instead of 19?



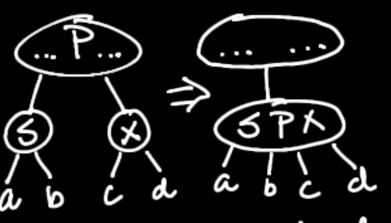
deleting is tricky

⑥ Remember: Delete from the leaf. The goal: ensure the leaf is a 3 or 4-node
As you search for the value and inorder successor, look and repair

- if a non-root 2-node, borrow from nearest sibling if the sibling is a 3 or 4-node

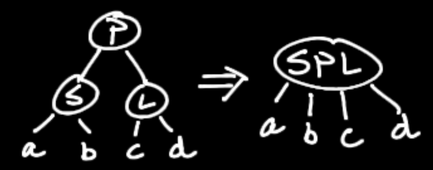


- But if the nearest sibling is a 2-node, merge to form a 4-node



Parent always has a value for a merge

- For a 2-node root, merge into a 4-node if the children are 2 nodes

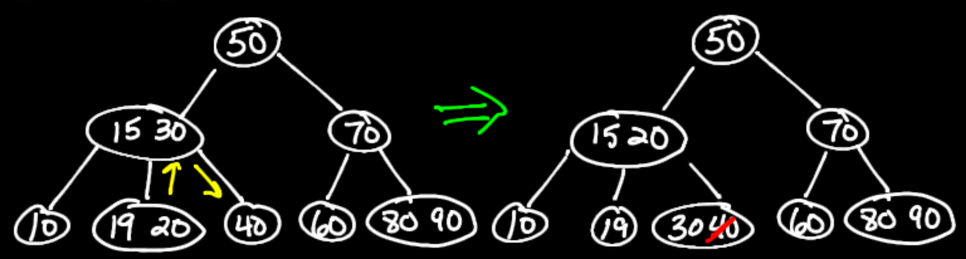


If they are not 2-nodes, you could borrow, meaning a merge is not needed

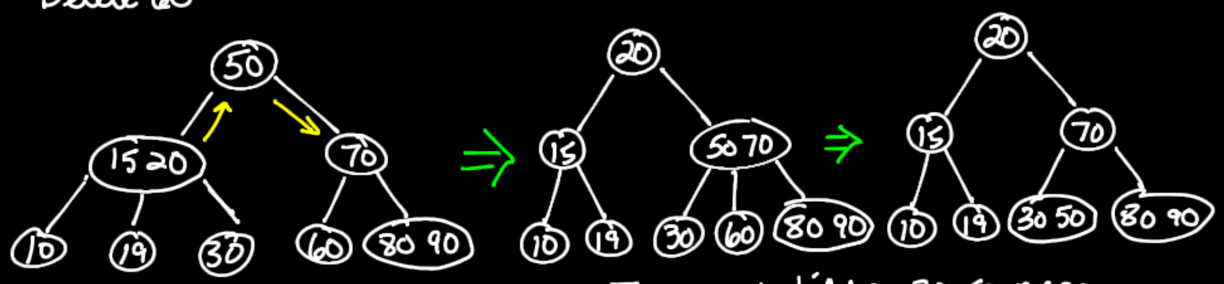
---

"nearest" sibling is the left sibling unless it is the left child, then it is its right sibling

⑦ Delete 40



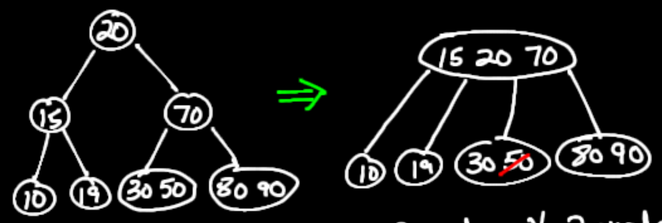
Delete 60



In this case, the root's child has 2 values, so don't merge, borrow when you get to 70

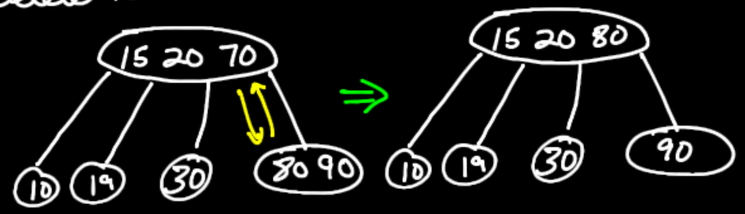
The nearest child is 30, so merge then we easily delete the 60

⑧ Delete 50

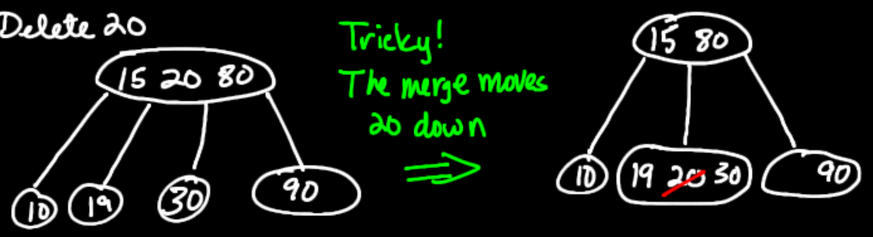


The root on the path is a 2-node w/ 2-node children, so merge and delete 50

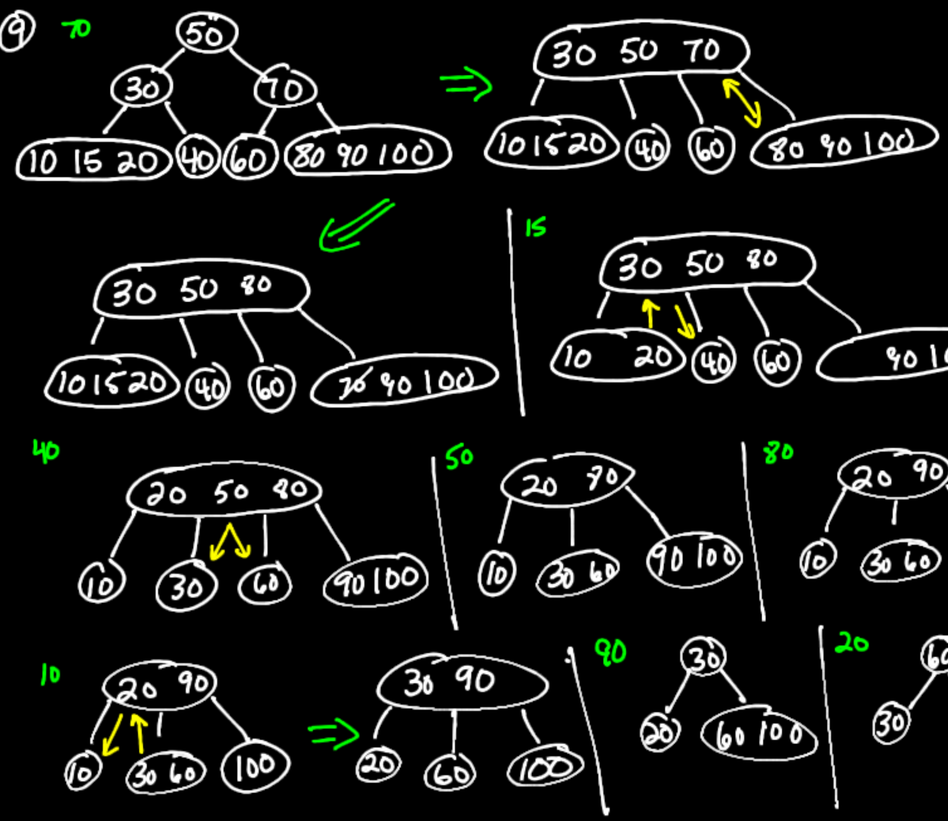
Delete 70



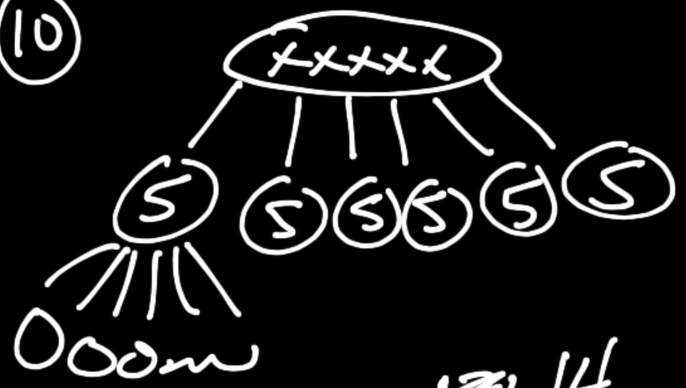
Delete 20



⑨ 70



10



13 14

